Lectern Design Spec: Kokoro-Narrated Video Lessons

Oct 18, 2026 · @Chase O'Neill

Summary

Lectern turns one Markdown lesson script into a narrated, verified video lesson and a VitePress page, regenerated from the repo with one command. Kokoro narrates, HyperFrames renders, and every technical claim traces to a pinned source.

Two pilot courses drive v1:

- **RAGAS:** self-study on RAG evaluation, monitoring, and regression tracking. You are the learner.
- **RAG reference stack:** teaching the team Haystack, Flask, Titan, GitLab CI/CD, WSL, Langfuse, and Tilt. You are the expert.

Design was locked in a grilling session on 2026-10-18. The engine spike passed all four tests the same day, and slice 1 is underway. Lectern is a working name, so rename it freely.

Goals and non-goals

Lectern exists to teach fast without ever teaching wrong. Every goal below is testable in the pilot.

Goals

- **Teach fast:** short lessons in a fixed skeleton, from Quick start to Recipes, with a 6-minute default cap.
- **Never teach wrong:** every claim traces to a pinned source, and every code block on screen actually ran.
- **Stay current:** lessons regenerate from source, and stale lessons say so on the page.
- **Serve every level:** skeleton entry points for everyone, plus personal variants from ELI5 to Internals.
- **Feel professional:** calm, consistent motion design that teammates take seriously.
- **Live in the wiki:** lessons land as searchable pages in your VitePress docs.
- **Cost nothing to run:** open-source parts only, local by default.

Non-goals for v1

- Paid services of any kind, including Remotion and hosted rendering.
- Languages other than English.
- Two-host dialogue or talking-head avatars.
- Generative imagery and stock footage.
- Learner tracking beyond anonymous variant counts.
- Team authoring and CI rendering. Both arrive in phase B.

Decision record

Twenty-one decisions are locked. Each row says what was decided, why, and what it commits the build to.

#	Decision	Why	Locks in
1	**Build model:** strict reproducible builds. One command regenerates every frame from the repo. Hand recordings enter only as pinned assets.	The stack will change. A frozen video keeps teaching the old way with the authority of a video.	Headless rendering, scripted capture, one container toolchain.
2	**Authors:** you alone until the house style is right. Component owners take drilldowns in phase B.	Five styles before one exists helps nobody.	Guardrails as code, so quality never waits on you.
3	**Where it runs:** your machine during the pilot. CI renders after the pilot, once runners check out.	Prove the build before automating it.	Same container and command in CI, so CI is a pipeline file, not a refactor.
4	**Skeleton:** Quick start, Map, Drilldowns, Recipes. Reference stays written. Each lesson has a 6-minute default cap, overridable.	One skeleton serves both courses. Engagement tops out around six minutes.	The Map is data with an ID per component.

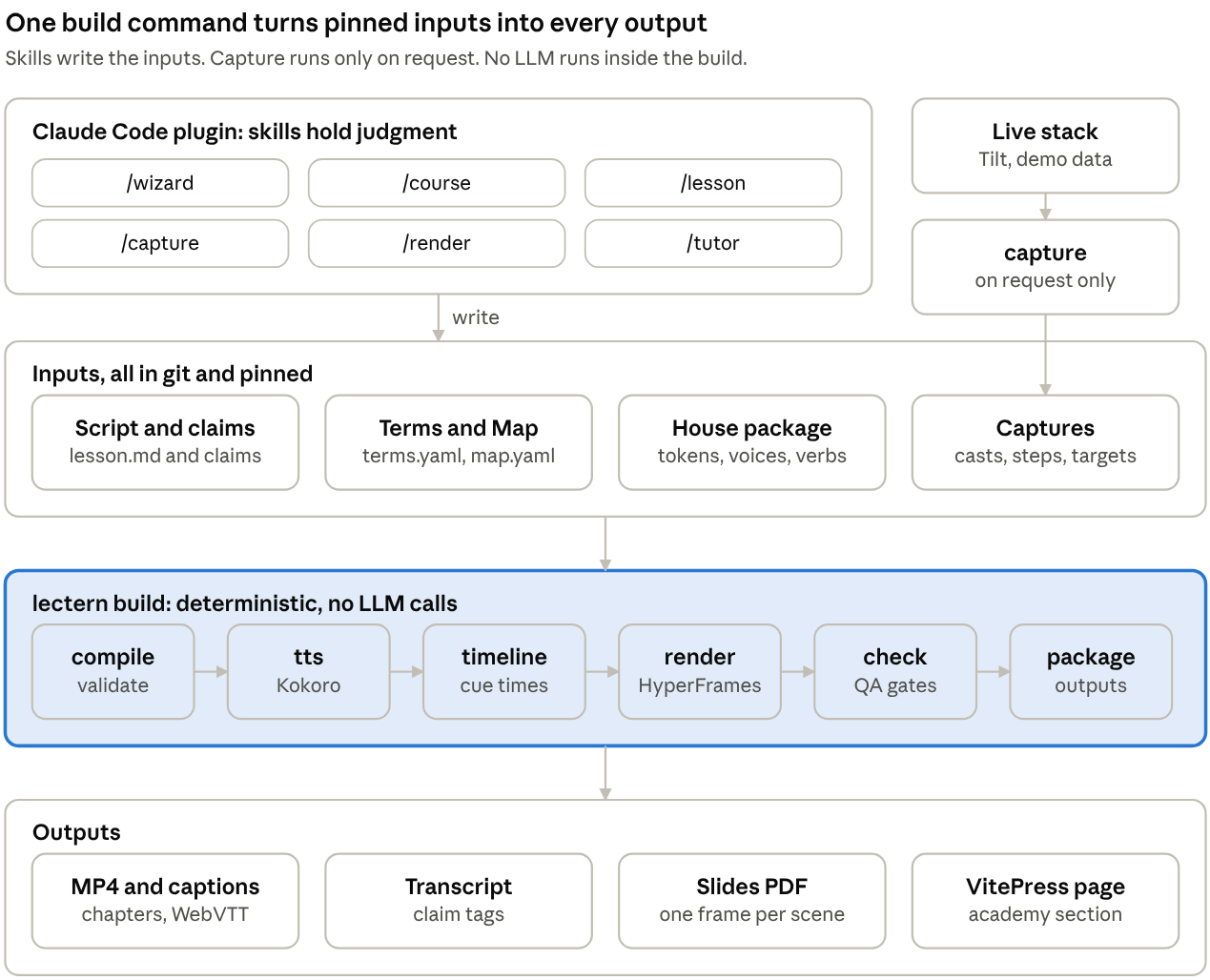
#	Decision	Why	Locks in
5	**Floors:** declared per course. Team: Python, Git, terminal. RAGAS: RAG literacy. Below the floor is a prerequisite.	People land mid-course from links, so lessons must stand alone.	One terms registry drives pronunciation, chips, and the glossary.
6	**Tutor mode:** five-rung ladder with placement by testing. Simplifying is free, deepening is gated. Variants stay local.	Self-rated level is the weakest signal. A learner cannot verify what they are learning.	A variant spec the professor hands off.
7	**Truth:** a claims ledger with source excerpts. Broken blocks, stale flags, a model judge advises. Two human gates in the pilot.	LLM scripts fail fluent, plausible, and wrong. RAGAS APIs churn.	Every code example executes during capture.
8	**Script:** one Markdown file per lesson with two tracks, word-anchored cues, claim tags, and checkpoints.	Anchors survive re-synthesis. Timecodes break on every edit.	Engine-agnostic scripts and a validating compiler.
9	**Voices:** Narrator and Examiner, picked by audition, shared with the professor.	A voice switch signals a mode change.	Voices locked in the house package, no per-lesson overrides.
10	**Engine:** HyperFrames behind a renderer adapter, pinned, spike first. No Remotion, no paid services.	Apache 2.0, deterministic, built for agent authoring.	A swap costs the adapter and templates, never a script.
11	**Look:** calm, diagram-first, dark. Design tokens, a closed motion vocabulary, a Map that holds still.	Consistent motion reads as meaning, not decoration.	A brand swap replaces tokens, not templates.

#	Decision	Why	Locks in
12	**Capture:** terminal as asciinema casts, browser as Playwright step captures with semantic targets. A redaction hit blocks the build.	Annotations land exactly, captures diff like code, and video seeking stops being load-bearing.	Raw video only as pinned assets.
13	**Outputs:** the lesson page comes first, with the MP4 as the portable copy. Exact captions, transcript, slides PDF, fidelity flag.	Video alone is the worst reference format.	Static output, no backend, no tracking.
14	**Packaging:** one Claude Code plugin with six skills and two agents, over a Python CLI in one container. No MCP.	Skills hold judgment, the CLI holds what must repeat.	No LLM call ever runs inside a build.
15	**VitePress:** generated pages plus a six-component theme, in an academy section of your wiki.	VitePress is the team's hosted docs of choice.	Pages are build output, never hand-edited.
16	**Repos:** tool, docs, RAGAS course, and a sibling repo for the stack course pinned to a stack commit.	Teams copy the stack as a template, so it must stay clean.	Staleness runs on a schedule, not inside the stack's merge requests.
17	**Pilot:** spike, RAGAS faithfulness drilldown, stack Quick start, stack Map, then the self-evolving plugin.	RAGAS first de-risks the pipeline without a live stack.	The phase B bar below.
18	**Wizard:** one skill gathers settings, overrides, auditions, and course or lesson intake through a guided grill.	Easy to start, hard to misconfigure.	One question file drives the skill and a terminal fallback.

#	Decision	Why	Locks in
19	**Narration cache:** one shared cache, on the render server during the pilot and in Git LFS after it.	Kokoro's audio can differ by one sample step between CPU models, which would change an approved MP4.	Every machine reuses the same audio, so a rebuild anywhere matches the approved cut.
20	**Self-evolving plugin:** a per-user layer over the official plugin, built last before phase B.	People will bend the tool's habits, and an update must not erase them.	Official and user versions are tracked apart, and the guarded rules stay on.
21	**Media beyond the academy:** one engine with a profile per output kind, verified claims in every kind, audience and intent fields, research briefs inside `/tutor`, and a theme per brand. All of it comes after the pilot.	The pilot proves the engine on two courses. These widen what it makes.	No pilot work changes, and every new kind reuses the claims checks and the gates.

Architecture

Lectern has three layers. Skills hold judgment, a deterministic CLI does every repeatable step, and one container carries the toolchain. No LLM call runs inside a build, so the same inputs always produce the same lesson.



Lectern pipeline · skills, inputs, six build stages, outputs

Skills write the inputs, capture refreshes pinned captures only when asked, and one build command runs six deterministic stages into every output.

Layers

- **Skills:** a Claude Code plugin. Six skills cover setup, planning, writing, capture, rendering, and tutoring. Two agents judge claims and inspect frames.
- **CLI:** Python. It owns config, synthesis, capture, rendering, checks, and packaging. `lectern build` runs the whole pipeline for one lesson.
- **Container:** CPU only, for amd64 and arm64. It carries Python, Kokoro with baked weights, Node 22 with HyperFrames, headless Chromium, FFmpeg, Playwright, the cast driver, and gitleaks.

Pipeline stages

1. **compile:** parse the script and validate cues, targets, claim tags, and terms. Emit a scene plan.
2. **tts:** synthesize each scene with Kokoro, applying registry pronunciations. Keep word timestamps.
3. **timeline:** map each cue anchor to its word's start time, add pauses and breathing room, and emit timeline.json.
4. **render:** the adapter turns timeline.json, templates, and captures into HyperFrames compositions. Headless Chrome seeks each frame and FFmpeg encodes.
5. **check:** run the deterministic gates.
6. **package:** write the MP4 with chapters, captions, transcript, slides PDF, and VitePress pages.

Capture sits outside `build`. It needs the live stack, so it runs only when you ask, and its outputs become pinned inputs to every later build.

Scene cache

Each scene renders under a hash of its inputs: narration text, voice and speed, the registry entries it uses, the captures it references, and the template and token versions. Edit one sentence and only that scene re-synthesizes and re-renders. Everything else comes from cache.

Data model and file formats

Every input is a text file in git, except screenshots, which go through Git LFS. Config layers in four levels: house package, course manifest, lesson front matter, then CLI flags.

Course manifest

`course.yaml` declares the floor, sources, pins, skeleton, and course defaults.

```
id: stack
title: RAG Reference Stack
kind: stack              # stack or topic
floor: [python, git, terminal]
prerequisites: [rag-foundations]
sources:
  stack:
    repo: <stack repo URL>
    commit: <pinned sha>
```

```
pins:
  python: "<version>"
  haystack: "<version>"
skeleton:
  quickstart: quickstart
  map: map
  drilldowns: [<component lessons in dependency order>]
  recipes: [<task lessons>]
map: map.yaml
terms: terms.yaml        # overlays the house terms
owner: chase
defaults:
  cap_minutes: 6
  fidelity: full
```

Lesson script

`lessons/<id>.md` holds narration and cues in one file.

```
---
id: ragas.faithfulness
type: drilldown
rung: practitioner
pins: {ragas: "0.4.3"}
captures: [run-faithfulness]
---

## concept  layout=diagram  diagram=faithfulness
Faithfulness asks one question. Is every claim in the [answer]{light}
backed by the retrieved [context]{light}? ^faith-01

## code  layout=code  src=examples/faithfulness.py
[These four lines]{lines=12-15} score one answer. ^faith-02
Here is the [real score]{play=run-faithfulness} from that run. ^faith-03

> ask: An answer scores a perfect 1.0 on faithfulness and is still wrong.
How?
```

- `## <scene> layout=<template>` opens a scene, with template parameters after it.
- Prose is the narration track.
- `[span]{verb=target}` fires a cue when the span's first word starts. A bare `{verb}` targets the node the span names.

- `^claim-id` ends every technical sentence.
- `> ask:` is a checkpoint.
- A cap override needs both `cap_minutes` and `cap_reason` in front matter.

Claims ledger

`lessons/<id>.claims.yaml` stores each claim's source and exact excerpt.

```
faith-01:
  kind: doc               # doc, code, or run
  source: {repo: ragas, ref: v0.4.3, path: <docs path>}
  excerpt: "<exact text copied from the pinned source>"
faith-03:
  kind: run
  source: {capture: run-faithfulness}
  excerpt: "<exact output line from the captured run>"
```

Terms registry

`terms.yaml` exists once in the house package and once per course. Course entries overlay house entries at compile time.

```
wsl:
  say: W S L
  define: Windows Subsystem for Linux, which runs Linux inside Windows.
langfuse:
  say: <respelling you approved>
  phonemes: <set by the audition>
  define: <one-line definition>
  aliases: [Langfuse]
```

Map spec

`map.yaml` pins every component to a grid, so the Map never moves between lessons.

```
grid: [12, 6]
nodes:
  - {id: <component>, label: <name>, at: [2, 1]}
edges:
  - {id: <edge>, from: <component>, to: <component>}
```

Captures

File	Holds
`capture/<id>.py`	Playwright step script, or a terminal step list
`captures/<id>.cast`	asciicast v2 text with real output
`captures/<id>/steps/*.png`	2x screenshots, stored in Git LFS
`captures/<id>/targets.json`	target ID, step, and box for every element touched
`captures/<id>/meta.json`	stack commit, paths shown, versions, redaction report

Variant spec

`variants/<lesson>.yaml` lives only on the learner's machine.

```
lesson: ragas.faithfulness
base: <base lesson build hash>
rung: foundations
focus: [faithfulness, context]
struggles: <the learner's own words>
```

House package

- `tokens.json`: color, type, spacing, and motion timing.
- `voices.yaml`: Narrator and Examiner voice IDs and speeds.
- `motion.yaml`: the twelve verbs with durations and easing.
- `terms.yaml`: house terms.
- `questions.yaml`: every wizard question.
- `templates/`: one scene template per layout.

Timeline and outputs

`timeline.json` is the engine-agnostic contract between the compiler and the adapter. It lists scenes with start and end in milliseconds, audio segments, and cues as verb, target, and time.

```
dist/<course>/<lesson>/
  lesson.mp4   lesson.vtt   transcript.md   slides.pdf
  page.md      timeline.json
  review/contact-sheet.pdf   review/qa-report.md
```

Narration

Narration carries meaning, the screen carries detail, and every sentence is written to be heard.

Script rules

1. **The voice carries meaning, the screen carries detail.** Code, paths, and on-screen text are never read aloud. That is Mayer's redundancy principle.
2. **Cues land on the word that names the thing.** That is temporal contiguity.
3. **Each visual change gets about one second of breathing room** before the next new idea.

The register rules from tts-speech-architect still apply: spoken phrasing, varied sentence length, and abbreviations spelled out on first use. Its data translation rule does not. In video, the screen shows the table and the voice narrates the insight.

Voice cast

- **Narrator** teaches. **Examiner** speaks only at checkpoints, so a voice switch means stop and think.
- Voices are picked by audition. The shortlist follows Kokoro's own grades: af_heart (A), af_bella (A-), af_nicole (B-), bf_emma (B-). The best male voices grade C+.
- The Examiner contrasts with the Narrator in accent or register.
- The cast is shared with the professor and locked in `voices.yaml`. Lessons cannot override it.

Pronunciation workflow

1. `compile` flags every word missing from Kokoro's dictionary and every acronym.
2. The wizard plays each new term in context. You accept it or give a respelling.
3. The resolved phonemes land in `terms.yaml`. Kokoro receives them through its inline phoneme syntax at synthesis.
4. Acronyms default to letters unless the registry says otherwise.

Scripts write numbers as words for the voice. Where a reader expects the written form, `[0.7]{say="zero point seven"}` splits the two: the voice and the captions say "zero point seven", while the checkpoint card, the page and the transcript show `0.7`.

Pacing

- Speed is set once per voice at the audition.
- Pause lengths between sentences and at scene breaks are tuned once in the pilot, then locked.
- At a checkpoint, the Examiner asks, a 5-second countdown runs on screen, and the Narrator answers.

Kokoro specifics

- Word timestamps cover English only. A missing timestamp falls back to the sentence start and raises a QA warning.
- There is no SSML or emotion control. Emphasis comes from sentence shape and punctuation.
- Weights are baked into the image, so synthesis needs no network.
- Output is 24 kHz mono, normalized to -16 LUFS.

Visual system

Every frame comes from HyperFrames templates driven by timeline.json, styled by design tokens, and moved by a closed set of twelve verbs.

Engine and adapter

- HyperFrames, pinned, behind a renderer adapter. The adapter reads timeline.json, templates, and captures, and emits HyperFrames compositions.
- Telemetry is off, and HeyGen's cloud render stays off.
- A swap costs the adapter and templates, never a script.

Scene templates

Title, chapter, map, diagram, code, cast, steps, split with code beside a cast, and checkpoint.

Motion vocabulary

Authors compose these verbs and never invent new ones. The compiler rejects anything else, and only one accent shows at a time.

Verb	Effect	Targets
light	Focus a target and dim the rest	Map or diagram node, capture target
pulse	Brief emphasis	Any target
trace	Animate along an edge to show flow	Map or diagram edge
zoom	Scale the view to a target	Capture target, node
pan	Move the view between targets	Capture target, node
spotlight	Darken everything but a region	Capture target
arrow	Draw an arrow to a target	Any target
callout	Short label anchored to a target, six words at most	Any target
lines	Focus code lines and dim the rest	Code scene
diff	Morph code from one version to the next	Code scene
play	Run a segment of a capture	Cast or step capture
mark	Outline text a command printed. The newest is bright, older ones fade	Cast scene
chip	Show a glossary chip	A registry term

Durations and easing per verb live in `motion.yaml`.

Design tokens and theme

- `tokens.json` holds color, type, spacing, and motion timing. A brand swap replaces tokens, never templates.
- Video is dark to match terminals and IDEs. Captures run in each app's dark mode where one exists.
- No stock footage and no decorative imagery. Decoration costs attention, which is Mayer's coherence principle.

Legibility floors, checked in QA

- Body text at least 32 px and code at least 28 px at 1080p, tunable in tokens.
- Contrast at WCAG AA or better.

- At most 15 code lines and 25 words of prose on screen at once.

Map rules

- Positions are pinned in `map.yaml`. Auto-layout drafts the Map once, then positions freeze.
- Every lesson in a course draws the same Map. A drilldown opens on it with its component lit.
- A map scene takes `component=<id>` to open with that component lit, and in the Map lesson that scene becomes the component's chapter. Back-to-back map scenes share one Map with no fade between them.
- `trace` follows one edge, or a path of components such as `browser>app>llm`, crossing edges in either direction. A line crossed against its arrow lights with its arrowhead at the start, so the lit path points the way the flow goes.
- Each build records the Map it drew. Editing `map.yaml` drops every lesson that draws it out of gate two until it is rebuilt, and `lectern map check` compares every map scene with a still of the Map alone, pixel by pixel.
- Code highlighting uses VS Code grammars through Shiki, so code looks like the editor.

Capture

Captures are data, not footage: text casts for terminals and step screenshots with element boxes for browsers.

Terminal

- A driver runs real commands in a pseudo-terminal and writes asciicast v2 files.
- Typing is presentation, and output is always real. Idle time compresses to fit the narration.
- A typed line, prompt included, must fit the terminal's width, or the screen wraps it mid-word. `lectern capture` warns before it runs one, and a backslash carries a long command onto a second line under bash's `>` prompt.
- Cast scenes replay through asciinema-player, which seeks to any moment. Text stays crisp at any zoom, and colors come from tokens.
- A mark outlines text a command printed, such as a score, where it last shows on screen, and holds until the next command plays. The newest mark is bright, and older ones fade to a quiet outline, or out entirely when the newest carries trail=clear. The build fails if the terminal does not show that text at the cue's word.

Browser

- Playwright scripts drive the UI step by step. Each step saves a 2x screenshot plus a box for every element it touched.
- Motion is synthesized on top: the cursor glides to the box, the click ripples, the view zooms, and the next step crossfades in.
- Real in-app motion, such as streaming tokens, gets a short burst of frames.
- Elements are found by role and name, never by pixels. `targets.json` maps each target ID to its step and box.

Pinned raw assets

Some steps cannot run in the container, such as Windows-side WSL setup. Those appear as a code scene with no output, or as a hand recording pinned as non-regenerable with a review date.

Redaction

- Captures run against a demo environment with seeded fake data and throwaway credentials.
- Playwright masks sensitive elements by locator at screenshot time.
- gitleaks scans every cast and every page's text dump. A hit blocks the build.
- Capture credentials live in a local env file outside git. The wizard points to it and never reads it.

Staleness

Each capture records the stack commit and the code paths it shows. When those paths change upstream, the capture and every lesson using it flip to stale.

Truth and QA

A lesson ships only when every claim traces to a pinned source, every code block ran, the checks pass, and both human gates sign off.

Claims

- Every technical sentence carries a claim ID. Each claim stores its kind, source, and exact supporting excerpt.
- Kinds: doc is a page at a pinned version, code is a repo path at a commit, run is captured output.

- Claims about LLM output cite the captured run, never a general truth, because scores vary between runs.
- Pinned sources are cached locally, so checks run offline.

Broken blocks, stale flags

Condition	Result
Excerpt missing from its pinned source	Broken: build blocked
Code on screen fails to execute	Broken: build blocked
Redaction hit in a cast or page text	Broken: build blocked
Excerpt missing at the stack's HEAD, or a newer library release exists	Stale: lesson flagged, page banner
Paths shown in a capture changed upstream	Stale: lesson flagged, page banner

Deterministic checks in `lectern check`

- Unknown cue verb or target.
- Technical sentence without a claim tag.
- Term missing from the registry.
- Length over the cap without a `cap_reason`.
- Word cap, minimum text size, contrast, and code line limit.
- Loudness within 1 LU of -16 LUFS, with no clipping.
- Missing word timestamps, as a warning.

Model judges advise, never gate

- **claims-judge** scores how well each excerpt supports its claim. It is never the agent that wrote the script, and it is calibrated on planted bad claims before first use.
- **frame-inspector** reviews the contact sheet for collisions, overflow, and unreadable text.

Human gates in the pilot

1. **Gate one, before render:** the script and claims ledger side by side, judge scores, and each new term with an audio snippet.
2. **Gate two, before publish:** the render, a contact sheet with the frame at every cue, and the QA report.

Outputs and VitePress

Each lesson becomes a VitePress page with a synced player, and the MP4 travels anywhere a page cannot.

Per lesson

- **MP4:** 1080p at 30 fps, H.264 and AAC, with chapters from scenes.
- **Captions:** WebVTT built from Kokoro's word timestamps, exact with no speech recognition pass. Soft and off by default.
- **Transcript:** Markdown with claim tags, for grep and machine ingestion. Your own RAG stack can index its own training.
- **Slides PDF:** one settled frame per scene.
- **VitePress page:** frontmatter, the transcript split by scene headings, and component tags. A do-not-edit header marks it as build output.

Per course

- A course page in skeleton order, with durations, prerequisites, and stale status.
- The CourseMap as navigation. Click a component to open its drilldown.
- A glossary page generated from the terms registry.

Theme components

Component	What it does
LessonPlayer	Video, synced transcript, chapters, and seeking on scene anchors
Claim	Links each claim to its source at its pinned version
Checkpoint	Answer first, then reveal
Term	Glossary chip with its definition
StaleBanner	Shows when a lesson is flagged stale
CourseMap	The Map as clickable navigation

Behavior

- Scene headings make transcripts searchable in VitePress's built-in search. A hit lands on the scene, and the player seeks to it.

- Components follow the site's light and dark mode. The video stays dark.
- The theme pins to your wiki's VitePress major version.
- No backend and no tracking. Progress lives only in the viewer's browser.
- **Fidelity flag:** full motion by default. Slides mode renders settled frames with the same narration, much faster.

Media weight

A 15-lesson course runs to hundreds of megabytes of MP4. Committing that into the docs repo bloats every clone. The options are Git LFS in the docs repo or a media path on the same host outside the repo, decided with the host after the pilot.

Pilot preview

`lectern preview` serves a minimal local VitePress site with the theme, so every page is seen as it will ship before anything touches the wiki.

Tutor mode

Any lesson can be re-scripted for one learner at another depth, without ever shipping an unverified claim.

Depth ladder

ELI5, Foundations, Practitioner, Expert, Internals. Base lessons render at Practitioner. A variant re-scripts the same lesson at another rung, refocused on what the learner flagged.

Placement by testing

The intake asks what the learner struggles with, then places the rung with two or three diagnostic questions from the lesson's checkpoint bank. Self-rated level is the weakest signal available, because people overrate their understanding until asked to explain it. The learner can still override the placement.

Simplifying is free, deepening is gated

- Going down the ladder only re-explains claims the base lesson already verified. No review is needed.
- Going up adds claims. Each one traces to a checkable source: a repo path, a pinned doc page, or code the pipeline executes.
- A deeper variant carries an "unreviewed" badge until the author promotes it. The gate holds even when the author is the learner.

Reproducible like any lesson

The intake writes a variant spec. The variant's script is saved beside it, so it rebuilds through the normal `build`.

Privacy

Variants render locally and never publish. Only an anonymous count of simplify and deepen requests per lesson flows back, which shows the author where a base lesson misses.

Professor boundary

The professor owns the intake grill and its gap ledger. Until it ships, `/tutor` writes the same variant spec. In v1, you use tutor mode on RAGAS. Teammates get it after the pilot.

The wizard skill

`/wizard` is the front door. It inspects the repo, proposes every value, asks only what it cannot derive, and writes config through the CLI, so a bad value never lands.

Modes

Mode	Runs when	Gathers	Writes
Setup	First run, or on request	Container check, a render profile timed on this machine, tokens, cap, fidelity, then hands off to Audition	House config
Audition	Inside Setup, or anytime	Narrator and Examiner picks, speed per voice, pronunciation of new terms	`voices.yaml`, `terms.yaml`
Course	Starting a course	Kind, sources and pins, audience and floor, prerequisites, skeleton, Map components, capture needs	`course.yaml`, draft `map.yaml`
Lesson	Starting a lesson	Type, objective, claim sources, captures, rung, cap override and reason, checkpoint seed	Lesson front matter stub
Override	Changing any setting	Key, scope, value, and a reason where the key requires one	Config at the chosen scope

Mode	Runs when	Gathers	Writes
Status	Anytime	Nothing. It shows effective config, stale lessons, and open gates	Nothing

Question engine

- One question at a time, each with a recommended default and one line of why. Same rhythm as a grilling session.
- Enter keeps the current value. Typing `default` resets to the inherited value.
- Inspect before asking. Git remote and commit, Tiltfile resources, dependency pins, and library versions become proposals, not questions.
- Tappable choices through AskUserQuestion where the client supports it, plain text otherwise.
- Every answer validates through the CLI the moment it is given.
- Each mode ends with a diff of the effective config, confirmed before anything is written.

Course intake, the guided grill

1. Is this a stack you built or a topic you are learning?
2. Source: the repo and commit, or the topic plus the authoritative sources to pin.
3. Audience and floor, proposed from the course kind.
4. Prerequisites that sit below the floor.
5. Skeleton: a proposed lesson list to confirm or edit.
6. Map: components drafted from the Tiltfile or the topic, with names and edges to confirm.
7. New terms found in the sources: confirm each definition, then audition its pronunciation.
8. Capture needs: what runs where, the demo environment, and where credentials live.

Lesson intake

1. Lesson type, taken from its skeleton slot.
2. Objective: after this lesson, the viewer can do what?
3. The sources its claims will cite.
4. Captures needed, existing or new.

5. Rung, with Practitioner as the default, plus any cap override and its reason.
6. Checkpoint seed: the misconception this lesson must kill.

Auditions

1. Render one technical paragraph, dense with registry terms, in each shortlisted voice.
2. Open a local comparison page with one player per voice.
3. Pick the Narrator, then an Examiner that contrasts with it.
4. Hear each chosen voice at three speeds and pick one per voice.
5. Hear each unknown term in context. Accept it, or give a respelling and hear it again.

Shared question definitions

Every question lives once in `questions.yaml`. The skill renders it as conversation, and `lectern wizard` renders the same file as terminal prompts with Enter-to-keep, for anyone working without Claude.

```
- key: cap_minutes
  scope: [house, course, lesson]
  prompt: Maximum lesson length in minutes?
  default: 6
  validate: {type: int, min: 1, max: 30}
  reason_required_on_override: true
  help: Engagement tops out around six minutes. Overrides need a reason.
- key: stack.commit
  scope: [course]
  prompt: Which stack commit does this course teach?
  derive: git rev-parse HEAD in the stack checkout
```

Precedence

CLI flag, then lesson front matter, then course manifest, then house package. `lectern config explain <key>` prints the chain and names the layer that won.

Resumable and headless

- Wizard state lives in `.lectern/wizard-state.json`. An interrupted session resumes at the last unanswered question.
- `lectern init --defaults` takes every default with no questions and fails if a required value has none. It serves scripted setup and, later, CI.

Guardrails

- The wizard never writes config files directly. The CLI owns every write and every validation.
- Credentials never pass through the wizard. It points to a local env file and never reads it.

Packaging and repos

Lectern ships as one Claude Code plugin over a Python CLI in one container, with four repos splitting the tool, the docs, and the courses.

Skills

Skill	Job
`/wizard`	Front door: setup, audition, course and lesson intake, overrides, status
`/course`	Plans a course: skeleton, lesson list, Map spec, pins. Input is a repo, a topic, or one of your study guides
`/lesson`	Writes one script with claims and excerpts, then assembles the gate-one packet
`/capture`	Writes the cast and step scripts, then runs them against the live stack
`/render`	Builds, runs checks and judges, then assembles the gate-two packet
`/tutor`	Learner intake into a variant spec, until the professor takes over

Agents

- **claims-judge:** scores excerpt support. It is never the agent that wrote the script.
- **frame-inspector:** checks the contact sheet for collisions, overflow, and unreadable text.

CLI commands

Command	Does
`init`	Builds the image and writes house defaults. `--defaults` skips every question
`wizard`	Terminal version of the wizard, from the same question file
`config`	Get, set, explain, and validate across all four layers
`audition`	Renders voice and term comparisons
`capture`	Runs capture scripts against the live stack
`build`	Compile, tts, timeline, render, check, and package for one lesson
`compile`, `tts`, `render`, `check`, `package`	Single stages, for debugging
`stale`	Compares pins against upstream and flags stale lessons
`preview`	Serves the local VitePress preview

Container

CPU only, built for amd64 and arm64. Kokoro weights are baked in, nothing downloads at render time, and Node runs only inside the render adapter.

Repos

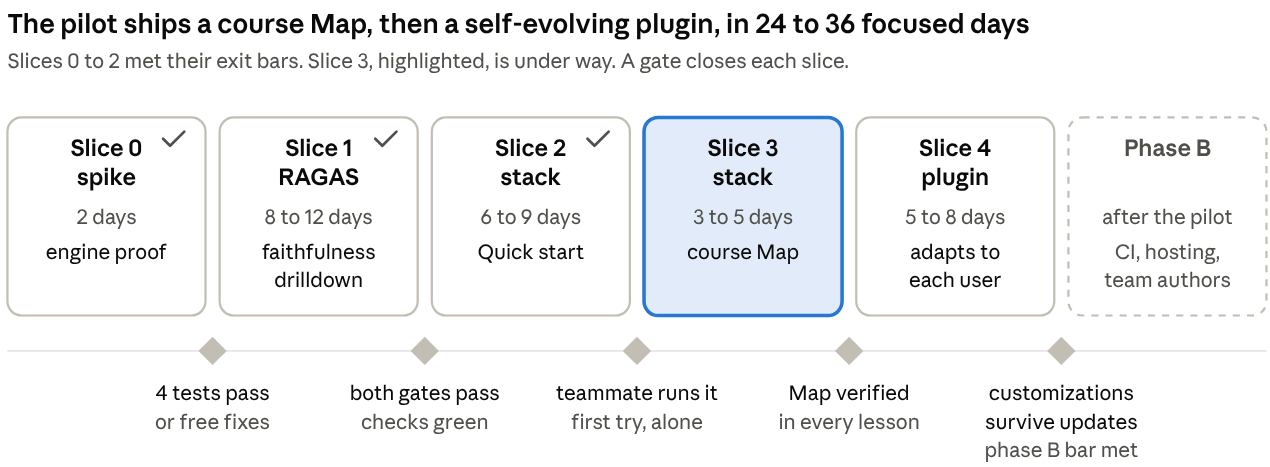
Repo	Holds
Tool	Plugin, CLI, container, VitePress theme, house package
Docs, your VitePress wiki	Generated academy pages only
RAGAS course	Scripts, claims, runnable examples, casts, the RAGAS pin
Stack course, a sibling repo	Scripts, claims, capture scripts, captures in Git LFS, the stack commit pin

Pins and reuse

- Each course pins a tool version, its library versions, and its source commits.
- Course terms overlay house terms at compile time.
- tts-speech-architect stays the tool for audio-only output.
- Output from technical-study-guide-architect is a valid input to `/course`.
- No MCP server. Claude Code already runs the CLI, so a server only makes sense for use without a shell.

Pilot plan

Five vertical slices take Lectern from engine spike to a navigable course Map, then to a plugin that adapts to each user, in an estimated 24 to 36 focused days. Each slice ships something real, and estimates get recalibrated from logged hours after slice 1.



Pilot roadmap · five slices, exit gates, phase B

Durations are estimates in focused days. Each diamond is an exit gate that must pass before the next slice starts.

- **Slice 0: spike, 2 days.** A container with HyperFrames, Kokoro, and FFmpeg runs four tests:
  - a capture plays frame-accurately under a zoom and a spotlight
  - a cue lands within one frame of its word
  - a 6-minute 1080p lesson renders in acceptable time on CPU
  - telemetry is off or can be disabled

  Exit: all four pass, or each failure has a free fix. Passed on 2026-10-18, natively and offline in the container. Every capture frame was exact, cues landed within 16.7 ms, a 6-minute lesson rendered in about 12 minutes on 2 CPUs, and nothing connected outward.
- **Slice 1: RAGAS faithfulness drilldown, 8 to 12 days.** It builds:
  - the compiler, tts, timeline, and renderer adapter
  - templates for title, diagram, code, cast, and checkpoint
  - the verbs light, pulse, lines, play, chip, and callout
  - the cast driver, claims ledger, excerpt checks, registry, audition, and gitleaks
  - every per-lesson output, plus LessonPlayer, Claim, Checkpoint, Term, and the local preview
  - `/wizard` for setup, audition, course, and lesson, plus `/course`, `/lesson`, `/render`, and claims-judge

Exit: the lesson passes both gates with every check green. Passed on 2026-10-18. The 1:56 faithfulness drilldown cleared gate one with 22 of 22 excerpts verified and every cited sentence judged 2 or higher by a calibrated claims judge, then gate two with every check green. Two cold renders gave identical frames, and editing one sentence re-rendered only its scene. Built cold and offline in the container, the lesson matched the native render frame for frame, with byte-identical Kokoro audio.

- **Slice 2: stack Quick start, 6 to 9 days.** It adds:
  - the sibling repo with its stack pin, and the demo environment
  - Playwright step captures with targets, bursts, masks, and synthesized cursor, zoom, and crossfade
  - the steps template and the verbs zoom, pan, spotlight, and arrow
  - `/capture`, frame-inspector, StaleBanner, and `stale`

  Exit: a teammate new to the stack reaches a working system on the first try, alone. Passed on 2026-10-18 on a zero-spend demo twin of the stack, since the real one can't be shared. The twin is public at github.com/chase-cto/rag-demo-stack. Three fresh agents, each a new teammate with only the lesson, reached a working system on the first try: once warm, then twice on a cold Docker cache behind a TLS-inspecting proxy. A fourth, on the final 3:55 cut, had every resource green in 77 seconds when it was stopped. Every capture runs a clean checkout of the pinned tag and 52 claims check against it. Two cold renders of the same inputs matched frame for frame, and so did a cold, offline build in the container. Kokoro audio matched byte for byte on one CPU model. Across CPU models, 9 of 46 sentences differed by one step in about 0.2% of their samples, so machines that build the same lessons share the TTS cache.
- **Slice 3: stack Map, 3 to 5 days.** It adds `map.yaml`, the map template, the trace verb, CourseMap, the course page, and the glossary page.
  Exit: the Map renders identically in every lesson and navigates to each drilldown. Met on 2026-10-19. All eleven map scenes across the Quick start, the 3:49 Map lesson and the 5:10 drilldown match a still of the Map alone, pixel for pixel. On the course page, the app opens its drilldown, Inside the app, and the other six components open their chapters in the Map lesson with the video at the chapter's start. Two cold renders of the Map lesson gave 6,859 of 6,859 identical frames. The claims judge needed four rounds on the Map lesson before no sentence scored below 2. On the drilldown, one fresh judge scored all 52 cited sentences 2 or more. Both gates were approved for all three lessons on stack v0.1.0. Building it found five engine fixes: code and terminal text drop font ligatures, a wrapped line aligns under the bracket its next line aligns to, a

lesson that starts where the Quick start left off points back to it, lectern lesson new asks a drilldown for its component, and a cut over 15 MB plays on its review page from a labelled re-encode while the approval pins the original.

- **Slice 3b: embedder drilldown and two recipes.** It adds:
  - rag-demo-stack v0.2.0: pgvector 0.8.7 as a switch, with search in the app's memory still the default, and retrieval knobs whose defaults keep v0.1.0's behavior
  - three tools that measure those knobs at no cost: a probe of the embedder, an evaluation over 31 docs and 71 labelled questions, and a benchmark of exact search against an HNSW index on 100,000 vectors
  - the drilldown Inside the embedder, and the recipes Measure retrieval and Switch to pgvector
  - the mark verb, which outlines text a command printed
  - the course re-pinned to v0.2.0, so every lesson is recaptured, rebuilt and sent back through both gates

  Exit: all six lessons pass both gates on stack v0.2.0. Met on 2026-10-19. All six build with every check green, and both gates are approved for all six. The claims judge took up to six rounds on a lesson before no sentence scored below 2. All 12 map scenes across four lessons match the Map still, and the course page opens Inside the embedder from the embedder box. Fresh frame inspectors found no major issue on the final cuts. Building it found four engine fixes: a mark fits its row with its border in the gap between characters, a newer mark takes the accent while older ones fade, a wrapped code row never starts left of its own line's indent, and front matter ends only at a line of exactly three dashes. The fade cleared four major findings in the pgvector benchmark scene, where six identical outlines had piled up. The dash rule fixed a preview that broke on a rule a command printed into a claim.
- **Slice 4: self-evolving plugin, 5 to 8 days.** The last layer before phase B. It adds:
  - a user layer kept outside the plugin, so a plugin update never overwrites it, with one record per customization: what it changes, why, when, and the plugin version it was made against
  - two versions in the metadata, the official plugin's and the user layer's own, both shown by `lectern status`
  - in-session capture: when you ask a skill to do something differently, it offers to make the change permanent and records it
  - guidance on every change, grounded in the house rules, with a clear recommendation for or against. Verified claims, both gates and the legibility floors can't be switched off

    - a grill for larger changes, to reach a best-practice version of what you want
    - on update, the user layer is re-applied to the new version, and any customization whose target changed is flagged and walked through with you

  Exit: three customizations survive a plugin update, and every conflict is surfaced and resolved with you.

After the pilot come `/tutor` and variants, the diff verb, the chapter and split templates, and the remaining lessons of both courses.

After the pilot: media beyond the academy. The first grill settled the shape, recorded as decision 21. The details get their own grill when phase B starts.

- **One front door.** `/lectern:new` asks what you are making, who it is for and why, then hands off to the skill that makes it. It is an ordinary plugin skill, so it needs nothing special from Claude Code.
- **One engine, more kinds of output.** Each kind is a profile on the same pipeline: frame size, length cap, burned-in captions, a hook and an end card. Social clips for LinkedIn and product demos come first.
- **Verified claims in every kind.** Promo copy is where overclaiming happens, so a clip's claims pass the same checks as a lesson's.
- **Audience and intent fields.** `audience: {who, knows, wants, rung}` and `intent: teach | inform | brief | demo | sell`. The intent picks the structure: teaching gets checkpoints, selling gets a hook and a call to action, and a brief puts the answer first. The rung stays the depth scale, with practitioner as the default for teaching.
- **Intents that scrutinize, not only teach.** Proposed, names still to settle. `assess` measures a system's current state against the state you want, shows where and why it falls short, and ladders the options for closing the gap. `debate` argues a position against its strongest counter-case. Every number comes from a captured run, repeated enough to show its spread, so a figure such as a recall rate of 35 to 37 percent is measured, never estimated. A grill fixes the goal, the metric and the target before anything runs. Example: a RAG engineer's pipeline that attributes quotes to speakers across sources and formats, measured with RAGAS.
    - **Secrets, settled.** Lectern sets the redaction policy, and it is strict by default. A video never shows a secret's value, even when someone asks for it. It describes what the secret is for, such as "API keys are set here and database passwords there", and gives the guidance anyway. Assessments are for one person, or a small

group of up to two teams working together. When an assessment is shared, its videos redact and generalize, and the page gives a script the person runs in their own terminal to look up the real values, so no agent ever sees them.
    - **Measurement, settled.** Lectern runs the measurement itself, as a capture of the eval harness, so its numbers are verified claims. It then compares its findings with your own evals and points out where they differ, including flaws in the evals themselves.
    - **Positions, settled.** Either side can set the position. You can state one and ask Lectern to test it against the evidence, let the evidence choose, or do both: argue your position, report what the evidence shows, and defend or revise either side.
- **Research briefs live in `/tutor`.** A brief is a tutor variant you make for yourself, from a topic, repo, dataset or process.
- **A theme per brand.** It is a layer between the house package and the course that holds a brand's tokens, logo and assets, taken from a style guide or a short description. The legibility and contrast floors stay on.

Acceptance bar for phase B

Phase B starts only when all five conditions hold.

- [ ] A teammate who has never run the stack follows the Quick start alone and reaches a working system on the first try.
- [ ] Editing one sentence re-renders only its scene.
- [ ] Two builds of the same inputs on the same machine produce identical frames.
- [ ] Zero lexicon misses and zero unverified claims ship.
- [ ] Your hands-on minutes per lesson are logged and trend down, against a target set after lesson three.

Phase B then unlocks CI rendering, the host and media store, and team authorship of drilldowns.

Defaults

Every default below ships in the house package. The last column says where it can be overridden.

Setting	Default	Override at
Lesson cap	6 minutes	Lesson, with `cap_reason`
Video	1080p, 30 fps, H.264 and AAC	House
Captions	WebVTT, soft, off by default	Viewer
Loudness	-16 LUFS	House
Music and sound effects	None	House
Title card	3 seconds, no outro	House
Theme	Dark video, page follows the site	Tokens
Body text	At least 32 px at 1080p	Tokens
Code	At least 28 px, at most 15 lines on screen	Tokens
Prose on screen	At most 25 words	Tokens
Breathing room	About 1 second after a visual change	`motion.yaml`
Checkpoint	Examiner asks, 5-second countdown, Narrator answers	Lesson
Acronyms	Spoken as letters	`terms.yaml`
Fidelity	Full motion	Build flag
Variant rung	Practitioner	Variant spec
Container	CPU only, amd64 and arm64, weights baked, no network at render	None
Engine	HyperFrames 0.8.170 pinned, telemetry off, cloud render off, fonts shipped locally	None
RAGAS judge model	Bedrock through your existing account	Ollama for zero spend
Language	English	None in v1
Render workers	Timed per machine at setup, passed to HyperFrames explicitly	Machine profile

Setting	Default	Override at
Render quality	Draft for previews, standard for publish	Build flag

Open items

Ten items are still open. Two of them gate the first commit.

Item	Owner	When
Settle where the tool repo lives and who owns it	Chase	Before the first commit
Confirm or replace the working name Lectern	Chase	Before the first commit
Pick the Narrator and Examiner voices	Chase	Slice 1 audition
Design the stack demo environment: seeded data, throwaway credentials	Build	Slice 2
Choose how to capture GitLab without real credentials: a sandbox project with a bot account, or a local GitLab container	Chase	Slice 2
Time a full lesson on a 4-CPU and an 8-CPU machine, and run the arm64 build	Chase	Before phase B
Pick the host and where MP4s live: Git LFS or a media path outside the repo	Chase	After the pilot
Move the shared narration cache into Git LFS, on GitHub or GitLab	Chase	After the pilot
Detail media beyond the academy: each output kind's profile, the audience fields, the theme layer's inputs, and the /lectern:new front door	Chase	After the pilot
Agree the variant spec handoff with the professor	Chase	When the professor ships

Risks and mitigations

The biggest risk is the engine: HyperFrames is young and there is no paid fallback. Every risk below has a named mitigation.

Risk	Mitigation
HyperFrames is pre-1.0 with no paid fallback	Pin the version and keep the adapter thin. The spike passed all four tests on 2026-10-18
Kokoro returns no timestamp for some words	Fall back to the sentence start and flag it in QA
English only	Accepted for v1
The model judge grades too leniently	Calibrate claims-judge on planted bad claims. Judges advise, gates gate
UI changes break step captures	Semantic locators, stale flags, and re-capture with one command
Motion polish eats weeks	Closed verb set, templates before polish, and slides mode as a floor
Corporate review of open-source parts	A license inventory in the tool repo and a container built from pinned sources
Ownership of the tool is unclear	Settle repo location and ownership before the first commit. Your employment agreement governs it, and this spec is not legal advice
Video weight bloats the wiki	Git LFS or an external media path, decided with the host

References

Pages consulted for this spec

- HyperFrames repository: license, rendering model, CLI, Claude Code plugin.
- HyperFrames open-source release coverage: April 2026 release date.
- HyperFrames version notes: pre-1.0 version, frame adapters.
- HyperFrames overview: positioning against Remotion.
- Remotion License FAQ: headcount threshold and pricing.
- Remotion Agent Skills guide: the rejected alternative's agent tooling.
- Remotion, Motion Canvas, and Revideo comparison: licenses and adoption.
- Remotion alternatives in 2026: Revideo folding into a commercial product.

- Motion Canvas package health: last release and maintainer count.
- Kokoro-82M voice grades: the audition shortlist.
- Kokoro word timestamps example: token start and end times, English only.
- asciinema player API: programmatic seek.
- Ragas v0.3 to v0.4 migration guide: the API churn the claims ledger guards against.
- Ragas release history: version pins.

Background, not re-read for this spec

- Ragas v0.1 to v0.2 migration guide.
- Guo, Kim, and Rubin, How Video Production Affects Student Engagement, 2014: the six-minute engagement finding.
- Mayer, multimedia learning principles: redundancy, coherence, and temporal contiguity.
- Rozenblit and Keil, 2002: the illusion of explanatory depth.
- Diátaxis documentation framework: the basis of the course skeleton.